HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Office for International Study Programs

Introduction to System on Chip
CO3143

Lab Report

Week 3: Single Cycle Datapath

Lecturer: Phạm Kiều Nhật Anh
Class: CC02
Student: Nguyễn Viết Tấn Vương - 2353345

Ho Chi Minh City, 2025

Contents

1 Introduction 2

2 System Implementation 2

3 Register File Design 2

4 Datapath and Control Unit 2

5 Waveform Analysis 3

1 Introduction

In this lab, I design a complete 32-bit Single-Cycle RISC-V Processor. The processor supports the RV32I base integer instruction set (Arithmetic, Logic, Load/Store, Branch, Jump) and the RV32M extension (Multiplication/Division).
The architecture consists of three main components:

1. **Register File:** Stores 32 general-purpose registers (x0-x31).
2. **Datapath & Control Unit:** Decodes instructions, executes ALU operations, handles branching logic, and manages data flow.
3. **Memory:** A unified memory module for both instructions and data (Harvard architecture simulation).

2 System Implementation

3 Register File Design

The Register File (`RegFile`) contains 32 registers of 32-bit width.

- **Read Operation:** Asynchronous. Data from `rs1` and `rs2` is available immediately. Register x0 is hardwired to 0.
- **Write Operation:** Synchronous on the rising edge of the clock. Writing occurs only if the write enable signal (`we`) is high and the destination register (`rd`) is not x0.

```
module RegFile (..., input clk, we, rst);
    reg [`REG_SIZE:0] regs[0:NumRegs-1];
    always @(*) begin
    rs1_data = (rs1 == 5'd0) ? 32'd0 : regs[rs1];
    rs2_data = (rs2 == 5'd0) ? 32'd0 : regs[rs2];
    end
    always @(posedge clk) begin
    if (rst) begin /* Reset logic */ end
        else if (we && (rd != 5'd0)) begin
        regs[rd] <= rd_data;
    end
    end
endmodule
```

Listing 1: Register File Implementation (DatapathSingleCycle.v)

4 Datapath and Control Unit

The `DatapathSingleCycle` module integrates the instruction decoding, ALU/CLA, Divider, and Control Logic.
1. Instruction Decoding: The 32-bit instruction is split into fields: `opcode`, `rd`, `funct3`, `rs1`, `rs2`, `funct7`. Immediate values for different instruction types (I, S, B, J) are sign-extended immediately.

2. ALU and Extensions: Instead of standard Verilog operators, the design instantiates the Carry Look-ahead Adder (CLA) from Lab 2 for addition/subtraction and a hardware **Divider** for division/remainder operations.
3. Control Logic: A large combinational `always @(*)` block acts as the Control Unit. It sets control signals based on `inst_opcode`, `funct3`, and `funct7`:

- **Branching:** Compares `rs1` and `rs2` (using CLA sum or direct comparison) to update `pcNext`.
- **Load/Store:** Calculates memory addresses and handles byte/half-word alignment using the CLA.
- **R-Type/I-Type:** Selects the correct operation (ADD, SUB, XOR, etc.) and routes the result to `wb_data` (Write-back data).

```
    always @(*) begin
        pcNext = pcCurrent + 32'd4;
        reg_we = 1'b0;

        case (inst_opcode)
            OpRegReg: begin
                reg_we = 1'b1;
                if (inst_funct7 == 7'd1) begin
                end else begin
                end
            end
        endcase
    end
```

Listing 2: Control Logic Example (DatapathSingleCycle.v)

5 Waveform Analysis

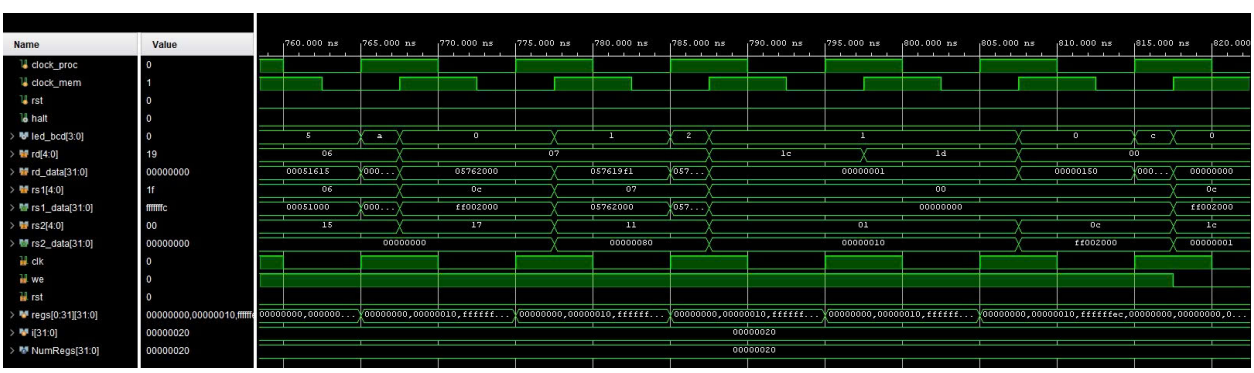

Figure 1: Simulation Waveform of the Processor Datapath

The waveform in Figure 1 illustrates the operation of the Register File and Datapath:

1. **Clock Signals:** `clock_proc` drives the processor, while `clock_mem` drives the memory (phase-shifted).
2. **Register Write:**

- At time 620ns: The write enable signal (`we`) goes high.
- Destination register `rd` is `0x02` (x2).
- Write data `rd_data` is `0xffffffec` (-20).
- At the rising edge of the clock, the register array `regs` updates, storing this value.

3. **Register Read:**
   - The signals `rs1_data` and `rs2_data` change immediately when `rs1` or `rs2` address changes (e.g., at 625ns, `rs1` changes to `0x02`, and `rs1_data` becomes `0xffffffec`). This confirms the asynchronous read behavior.
4. **Program Counter:** Although not explicitly expanded, the sequential execution implies the PC is updating correctly to fetch new instructions that drive these register changes.

— End —